



Document Prepared By **Hamdan Ali**



NAME : HAMDAN ALI

WEEK 7 : Day 35

DAY 35

TASK : Screenshoots Of Output

CODEBASE Of the Work

The screenshot displays a web application running on a local machine. The browser window shows the URL `http://localhost:5173/login` and a login form with the following elements:

- Welcome back**
- Log in to see your tasks.
- Email input field
- Password input field
- Log in** button
- Link: [Don't have an account? Create one](#)

The terminal window at the bottom shows the command `npm run dev` being executed in the directory `PS D:\internship (HindukushSoft Technologies)\Week7\taskflow-frontend`. The output indicates the application is running locally at `http://localhost:5173/` and provides instructions for network exposure and help.



Full PROJECT View

Welcome back

Log in to see your tasks.

Please correct the highlighted fields.

Email

demo@example.com

Password

.....

Log in

Don't have an account? [Create one](#)

Welcome back

Log in to see your tasks.

Unable to reach the server. Please check your internet connection.

Email

demo@example.com

Password

.....|

Log in

Don't have an account? [Create one](#)



EXPLANATION OF THE TASK

DAY 35

TASKFLOW

Day 35

Centralised Error Handling And Validation, Front to Back

Week 7 — Day 35

Task: Handle Backend and Frontend Errors Properly

Learning Topics: Error Handling, Validation

Stack: Laravel 12 (Exception Handling), React 18

1. Introduction And Objective

Day 35's objective was to handle backend and frontend errors properly — the learning topics were error handling and validation. Up to this point, individual pieces of validation and error handling already existed scattered across earlier days (Form Requests in Day 33, a 401 interceptor in Day 34). Day 35's job was to pull all of that into one deliberate, centralised, consistent strategy so that no matter what goes wrong — a bad input, an expired session, a missing record, or a genuine server crash — the user always sees a clear, predictable, human-readable message instead of a blank screen, a raw stack trace, or a silent failure.

I approached this the way a real engineering team would: define one response contract for errors, implement it once at the framework level on the backend, and implement one matching parser once on the frontend, so every feature built on top of it (forms, lists, auth) automatically benefits without repeating the same defensive code everywhere.

2. Backend: Centralised Exception Handling

2.1 The Problem with Handling Errors Ad-Hoc

Without central handling, every controller method would need its own try/catch, and a forgotten one anywhere in the app would leak Laravel's default HTML error page (or a raw, unstructured JSON blob) straight into what is supposed to be a strict JSON API. That inconsistency is exactly what a frontend developer (and any client) least wants to work against.

2.2 A Single Exception-to-JSON Translator

I implemented `app/Exceptions/Handler.php`, which registers one `renderable()` callback that intercepts every exception thrown anywhere in the application for any request under `/api/`, and maps it to a clean, consistent JSON payload with the correct HTTP status code, using a match expression to keep the mapping easy to read and extend:

Exception	Status	Message returned to the client
ValidationException	422	"The given data was invalid." + field-level errors
AuthenticationException	401	"You must be logged in to perform this action."
AuthorizationException	403	The specific reason access was denied
ModelNotFoundException / 404	404	"The requested resource could not be found."
Anything else (500, etc.)	as-is / 500	A safe generic message in production; the real message only when app.debug is true

```
[ $status, $message, $errors ] = match (true) {  
  $e instanceof ValidationException => [422, 'The given data was invalid.', $e->errors()],  
  $e instanceof AuthenticationException => [401, 'You must be logged in...', null],  
  $e instanceof AuthorizationException => [403, $e->getMessage() ?: 'No permission.',  
  null],  
  $e instanceof ModelNotFoundException, $e instanceof NotFoundHttpException => [404,  
  '...', null],  
  default => [ $e->getStatusCode() ?? 500, config('app.debug') ? $e->getMessage() :  
  'Something went wrong...', null ],  
};
```

A crucial security-conscious detail here: in production (APP_DEBUG=false) the raw exception message for unexpected 500-class errors is deliberately hidden from the client and replaced with a generic, safe message — internal details like database structure, file paths, or stack traces should never be exposed to an API consumer, only ever logged server-side.

2.3 Consistent Success Responses Too — the ApiResponse Trait

Centralising error responses is only half the job; success responses need the exact same envelope shape for the frontend's parsing logic to be genuinely universal. I extracted a small ApiResponse trait with success() and error() helper methods that any controller can use, guaranteeing that literally every response the API ever sends — success or failure — has the shape { success, message, data?, errors? }.

2.4 Field-Level Validation with Custom Messages

Building on the Form Requests introduced in Day 33 (StoreTaskRequest, UpdateTaskRequest), I made sure every validation rule has a specific, human-friendly message via the messages() method — instead of Laravel's default, more generic "The title field is required" style text — directly reflecting the guidance in the Laravel form-validation article I referenced this week (a title needs a value, a status must be one of a specific set of options, a due date can't be in the past, and so on).

3. Frontend: A Single Error-Parsing Utility

3.1 parseApiError — One Function, Every Error Shape Handled

Rather than writing err.response?.data?.message repeatedly (and inconsistently) throughout the codebase, every part of the frontend that talks to the API funnels its catch block through one shared utility, src/utills/parseApiError.js, which normalises any possible failure — a validation 422, an expired-session 401, a permissions 403, a missing-resource 404, a server 500, or even a request that never reached the server at all (offline, CORS, DNS, timeout) — into one predictable shape: { message, fieldErrors }.

```
export function parseApiError(err) {
  if (!err.response) {
    return { message: 'Unable to reach the server...', fieldErrors: {} };
  }
  const { status, data } = err.response;
  if (status === 422) return { message: data?.message, fieldErrors: data?.errors || {} };
  if (status === 401) return { message: 'Your session has expired...', fieldErrors: {} };
  // ...403, 404, 500+ handled the same predictable way
}
```

This single function is what makes it possible for TaskForm to show a validation message under the exact input that failed, while a list page shows a friendly banner with a "Try again" button,

and the auth pages show a session-expired notice — all from the exact same underlying utility, with zero duplicated parsing logic anywhere in the app.

3.2 Catching Render-Time Errors — the ErrorBoundary

A try/catch around an API call cannot catch a bug that happens while React is rendering a component (for example, calling `.map()` on something unexpectedly undefined). For that class of failure, React provides Error Boundaries — and I implemented one (`src/components/ErrorBoundary.jsx`) that wraps the whole application. If any component below it throws during render, the boundary catches it, logs it (in a real production deployment this is where you would forward the error to a monitoring service like Sentry), and shows a graceful "Something went wrong" screen with a recovery button — instead of the entire React app unmounting into a blank white page, which is what happens by default without a boundary.

3.3 Toast Notifications for Transient Feedback

For actions that succeed or fail outside the context of a specific form field (e.g. deleting a task, logging out), I added a lightweight `ToastProvider` so the user gets brief, non-blocking confirmation or error feedback without needing a full-page alert — this complements the inline field errors used on forms rather than replacing them.

4. Putting It All Together — An End-to-End Example

Consider a user submitting the task form with an empty title:

1. React sends `POST /api/tasks` with an empty title field.
2. `StoreTaskRequest`'s `rules()` rejects it; `failedValidation()` throws an `HttpResponseException` carrying a 422 JSON payload with the custom message "Please give the task a title."
3. Laravel's central Handler is bypassed here specifically because the Form Request already returns the correctly-shaped response directly — demonstrating that both layers (Form Requests and the global Handler) cooperate rather than conflict.
4. Axios rejects the promise; `TaskForm`'s catch block calls `parseApiError(err)`, which recognises status 422 and extracts the field-level errors.
5. The exact message "Please give the task a title." is rendered directly beneath the Title input — the user immediately knows exactly what to fix.

5. Challenges And How I Solved Them

- Deciding where validation "belongs" — Form Requests handle input validation (is this data well-formed?), while the central Handler handles everything else (is this even a valid route,

does this record exist, is the user authorised, did something crash?). Keeping this boundary clear avoided a tangle of overlapping error-handling code.

- Not leaking sensitive internals in production — solved by gating the raw exception message behind `config('app.debug')`, so local development still shows the real error for fast debugging while a production deployment never exposes it.
- Making sure a JavaScript rendering bug can never take down the whole app silently — solved with a top-level `ErrorBoundary`, which is easy to forget but essential for a production-grade React app.

6. Key Takeaways

- A REST API should have exactly one error response shape, defined once, applied everywhere — not per-controller ad-hoc handling.
- Form Request validation and a global exception handler are complementary layers, not competing ones.
- Never expose raw exception messages or stack traces to API clients in production.
- React error boundaries and API try/catch solve two different problems — a robust app needs both.
- A single, shared error-parsing utility on the frontend eliminates repetitive, inconsistent error-handling code across components.

WHERE TO FIND THE CODE

All source code for this task is included alongside this document inside the `backend/` and `frontend/` folders.

THANK YOU!